

Digital Search Tree & Trie

Prof. Siddharth Shah

Digital Search Tree (DST)

- A digital search tree (DST) is a binary tree whose ordering of nodes is based on the values of bits in the binary representation of a node's key (Knuth,1997).
- The ordering principle is very simple: at each level of the tree a different bit of the key is checked;
 - if the bit is 0, the search continues down the left subtree,
 - if it is 1, the search continues down the right subtree.
- The search terminates when the corresponding link in the tree is NIL.
- Every node in a DST holds a key and links to the left and right child, just like in an ordinary binary search tree.

Creating DST

- Create a DST by inserting following keys into an empty tree

A 00001

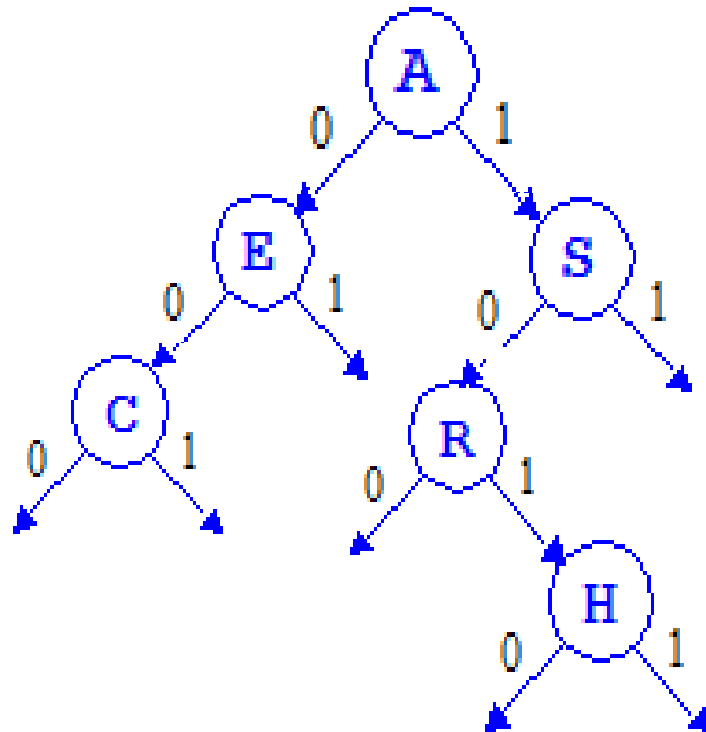
S 10011

E 00101

R 10010

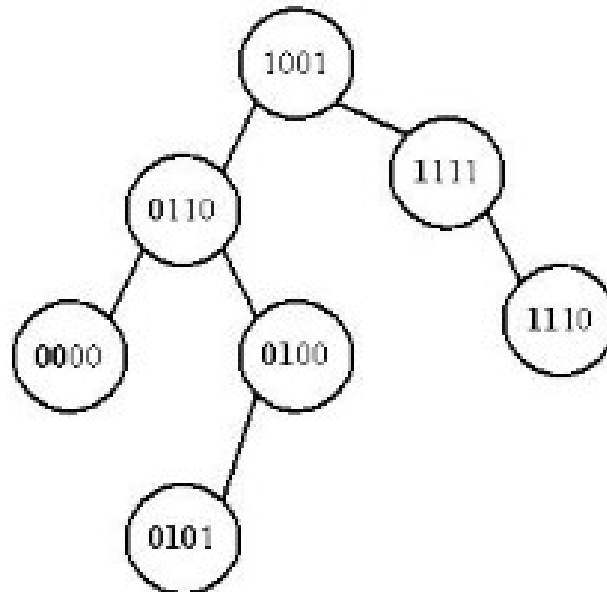
C 00011

H 10100



Deleting a node from DST

- Procedure to delete a node in the DST simply replaces the node with any leaf node in either of the node's subtrees.
- If the node to be deleted does not have any children, it is simply removed and replaced by a NIL pointer.
- For instance, removing the node **0110** from the tree in Figure below can be done by replacing it with either node **0000** or **0101**.



DST Properties

- In a DST, each key is somewhere along the path specified by the bits in the key (this guarantees that the find and insert algorithms will work)
- Suppose each key contains no more than B bits. Then:
 - The worst case height of a DST containing N keys is B
 - **Compare:** worst case height in a regular BST containing N keys is N, which with B-bit keys can be as much as 2^B
- So, when N is large and B is comparable to $\log_2 N$, DST's give worst-case guarantees comparable to balanced BST's, and are much easier to implement

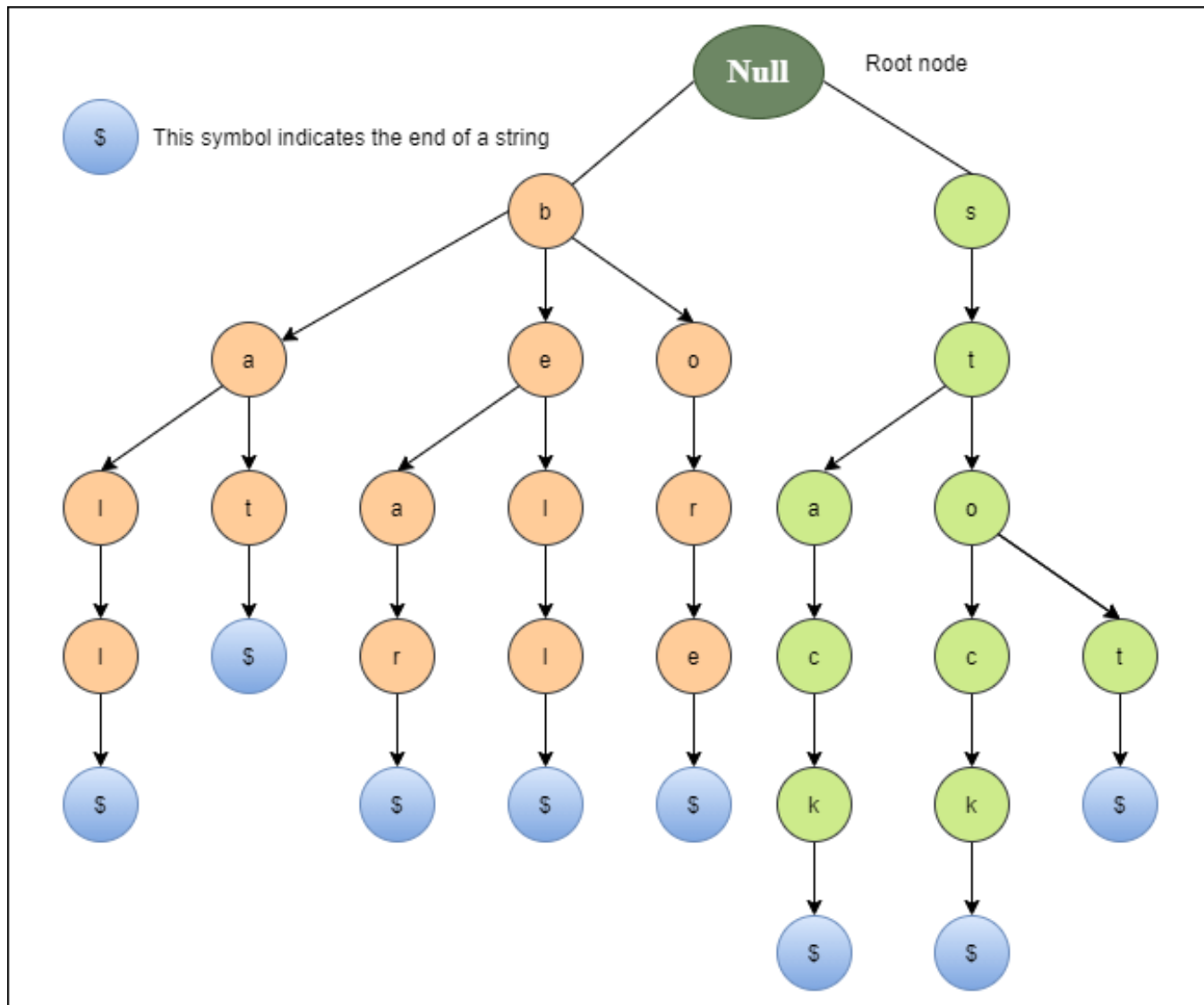
DST Properties

- However, DST's do not have a strong key ordering property:
 - The key in a node X can be larger or smaller than keys in either of its subtrees
 - So, an ordinary traversal of a DST is not guaranteed to visit keys in sorted order
- So consider tries:
 - Give strong key ordering property
 - Preserve the nice worst-case properties of DST's
 - Do true radix search, avoiding repeated full-key comparisons

Trie

- Trie is an efficient information retrieval data structure.
- Trie is a sorted tree-based data-structure that stores the set of strings.
- It has the number of pointers equal to the number of characters of the alphabet in each node.
- It can search a word in the dictionary with the help of the word's prefix.
- For example, if we assume that all strings are formed from the letters 'a' to 'z' in the English alphabet, each trie node can have a maximum of 26 pointers.

Trie



Trie Properties

- The root node of the trie always represents the null node.
- Each child of nodes is sorted alphabetically.
- Each node can have a maximum of 26 children (A to Z).
- Each node (except the root) can store one letter of the alphabet.

Creating Trie

- Inserting a key into Trie is simple approach. Every character of input key is inserted as an individual Trie node.
- Note that the children are pointers (or references) to next level trie nodes. The key character acts as an index into the children array.
- If the input key is new or an extension of existing key, it is needed to construct non-existing nodes of the key, and mark end of word for last node.
- If the input key is prefix of existing key in Trie, we simply mark the last node of key as end of word. The key length determines Trie depth.

Searching Trie

- Searching for a key is similar to insert operation, however here only compare the characters and move down.
- The search can terminate due to end of string or lack of key in trie.
 - In the former case, if the isEndofWord field of last node is true, then the key exists in trie.
 - In the second case, the search terminates without examining all the characters of key, since the key is not present in trie.

Applications of Trie

- Spell Checker
- Auto-complete

Analysing Trie

- Using Trie, search complexities can be brought to optimal limit (key length).
- If keys are stored in binary search tree, a well balanced BST will need time proportional to $M * \log N$, where M is maximum string length and N is number of keys in tree.
- Although in a balanced BST of N nodes you need to visit $O(\log N)$ nodes to find the targeted value in the worst case, it will also take time to compare a node's string with the target string. As the maximum length of a word is given as M , the worst case time complexity for each individual string comparison is $O(M)$. As such a comparison happens at each visited node, this gives a total complexity of $O(M \log N)$.
- Using Trie, the key can be searched in $O(M)$ time. However the penalty is on Trie storage requirements.